Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL602**	**Course Name:**	**Cryptographic and system security Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**03**
Title of the Experiment:	**Implementation of Diffie Hellman Key Exchange Algorithm**
Date of Performance:	**06/02/2026**
Date of Submission:	**13/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Ms Kshitija Gharat

Signature :

Date:

Aim: Write a program to implement Diffie-Hellman Algorithm.

Objectives:

• To understand the principles of symmetric key cryptography.

• To understand the Diffie-Hellman Key exchange algorithm.

• To understand the possible attacks on

Diffie-Hellman. Outcomes: The learner will be able to

Apply the cryptosystem to ensure secure key exchange between sender and receiver.

Hardware / Software Required: C/C++/JAVA/Python

Theory:

Diffie-Hellman key exchange is a cryptographic protocol that allows two parties that have no prior knowledge of each other to jointly establish a shared secret key over an insecure communications channel. This key can then be used to encrypt subsequent communications using a symmetric key cipher.

The Diffie–Hellman key exchange algorithm solves the following dilemma. Alice and Bob want to share a secret key for use in a symmetric cipher, but their only means of communication is insecure.Every piece of information that they exchange is observed by their adversary Eve.

How is it possible for Alice and Bob to share a key without making it available to Eve? At first glance it appears that Alice and Bob face an impossible task. It was a brilliant insight of Diffie and Hellman that the difficulty of the discrete logarithm problem for F* p provides a possible solution. The simplest, and original, implementation of the protocol uses the Multiplicative group of integers modulo p, where p is prime and g is primitive root mod p. Here is an example of the protocol:

Step 1: Alice and Bob get public numbers P = 23, G = 9

Step 2: Alice selected a private key a = 4 and Bob selected a private key b = 3

Step 3: Alice and Bob compute public values Alice: x =(9^4 mod 23) = (6561 mod 23) = 6

Bob: y = (9^3 mod 23) = (729 mod 23) = 1

Step 4: Alice and Bob exchange public numbers

Step 5: Alice receives public key y =16 and Bob receives public key x = 6

Step 6: Alice and Bob compute symmetric keys Alice: ka = y^a mod p = 65536 mod 23 = 9

Bob: kb = x^b mod p = 216 mod 23 = 9

Step 7: 9 is the shared secret.

In the original description, the Diffie-Hellman exchange by itself does not provide authentication of the communicating parties and is thus vulnerable to a man-in-the-middle attack. A person in the middle may establish two distinct Diffie-Hellman key exchanges, one with Alice and the other with Bob, effectively masquerading as Alice to Bob, and vice versa, allowing the attacker to decrypt (and read or store) then re-encrypt the messages passed between them. A method to authenticate the communicating parties to each other is generally needed to prevent this type of attack.

Algorithm:

Alice and Bob, two users who wish to establish secure communications. We can assume that Alice and Bob know nothing about each other but are in contact.

1. Communicating in the clear, Alice and Bob agree on two large positive integers, p and g, where p is a prime number and g is a primitive root mod p.
2. Alice randomly chooses another large positive integer, XA, which is smaller than p. XA will serve as Alice's private key.
3. Bob similarly chooses his own private key, XB.
4. Alice computes her public key, YA, using the formula YA = (g^XA) mod p.
5. Bob similarly computes his public key, YB, using the formula YB = (g^XB) mod p.
6. Alice and Bob exchange public keys over the insecure circuit.
7. Alice computes the shared secret key, k, using the formula k = (YB ^XA) mod p.
8. Bob computes the same shared secret key, k, using the formula k = (YA ^XB) mod p.
9. Alice and Bob communicate using the symmetric algorithm of their choice and the shared secret key, k, which was never transmitted over the insecure circuit

Implementation:

```
# Step 1: Publicly shared values
p = int(input("Enter a prime number (p): "))
g = int(input("Enter a primitive root of p (g): "))

# Step 2: Private keys
a = int(input("Enter Alice's private key (a): "))
b = int(input("Enter Bob's private key (b): "))

# Step 3: Compute public keys
x = pow(g, a, p)   # Alice's public key
y = pow(g, b, p)   # Bob's public key

print("\nAlice's Public Key (x):", x)
print("Bob's Public Key (y):", y)

# Step 4 & 5: Exchange public keys and compute shared secret
ka = pow(y, a, p)  # Alice computes shared secret
kb = pow(x, b, p)  # Bob computes shared secret

print("\nShared Secret Key computed by Alice:", ka)
print("Shared Secret Key computed by Bob:", kb)

# Step 6: Verification
if ka == kb:
    print("\nKey exchange successful! Shared secret is:", ka)
else:
    print("\nKey exchange failed!")
```

Output:

```
Enter a prime number (p): 23
Enter a primitive root of p (g): 9
Enter Alice's private key (a): 4
Enter Bob's private key (b): 3

Alice's Public Key (x): 6
Bob's Public Key (y): 16

Shared Secret Key computed by Alice: 9
Shared Secret Key computed by Bob: 9

Key exchange successful! Shared secret is: 9
```

Conclusion:

This experiment demonstrated how two parties can securely establish a shared secret key over an insecure communication channel without directly transmitting the key. . Overall, this study enhanced understanding of symmetric key cryptography and the importance of secure key exchange mechanisms in modern cryptographic systems.